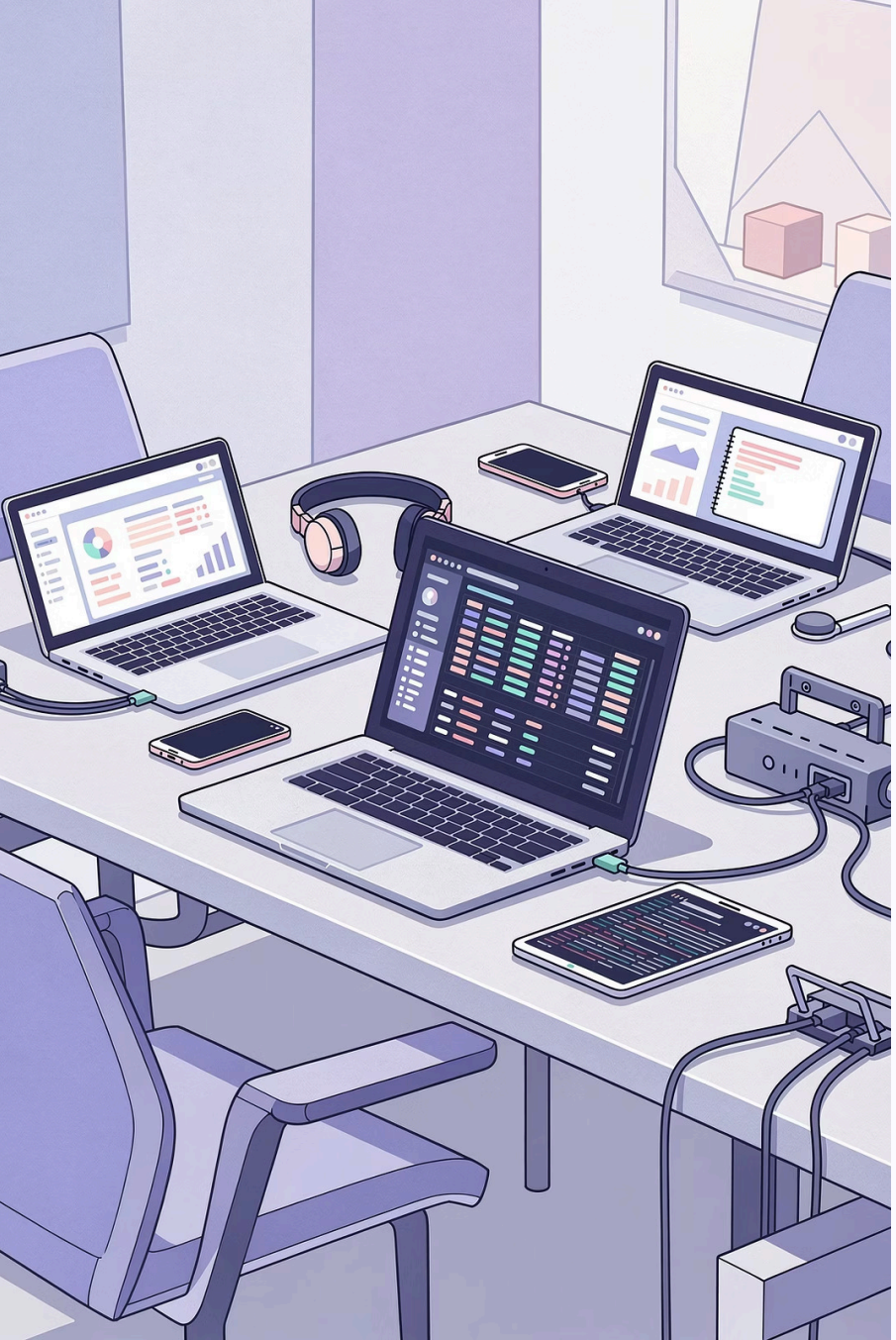
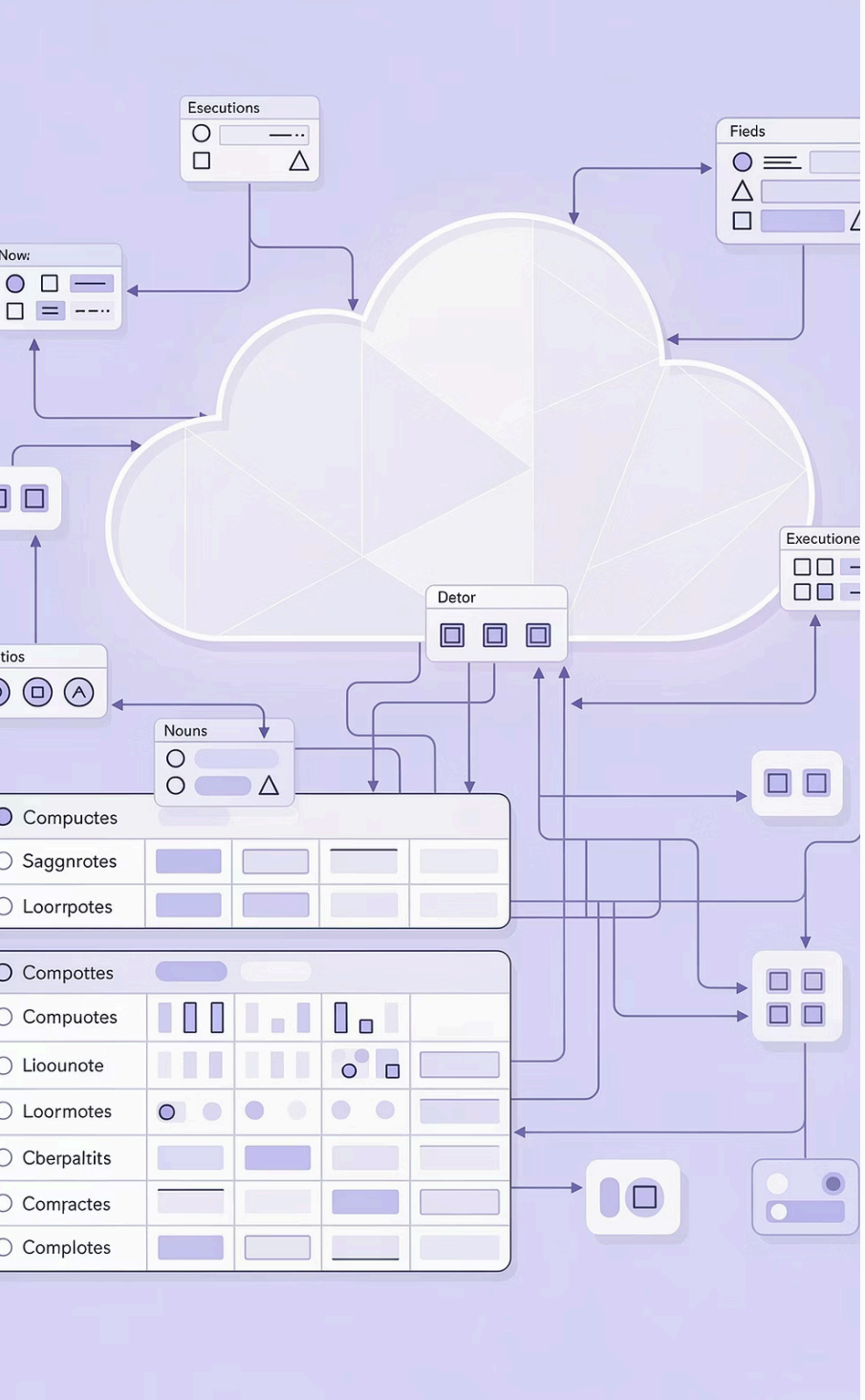


DATABRICKS TECHNICAL REFRESHER

# DataFrames in Databricks

DataFrames are the main abstraction for structured data in Spark and Databricks, letting you define transformations while the engine handles execution. They are lazily evaluated, turned into DAGs, and optimized by Catalyst before running at scale. In practice, they power common tasks like creating, filtering, shaping, and aggregating data before SQL-based workflows.




CHAPTER 1 FOUNDATIONS

# What a DataFrame Really Is

A DataFrame is more than a table-like object with named columns. In Databricks, it represents a **distributed dataset** plus the metadata and logic required to process it efficiently. Two ideas matter most: a **schema**, which defines column names and data types, and an **execution plan**, which captures the operations you want Spark to perform.

## Distributed by design

Data is partitioned across executors, so operations can scale beyond a single machine without changing the user-facing API.

## Schema-aware

Unlike unstructured collections, DataFrames preserve types and column semantics, enabling better validation and optimisation.

## Plan-driven

Your code describes intent. Spark records the steps, then decides the most efficient way to execute them later.

# Lazy Evaluation

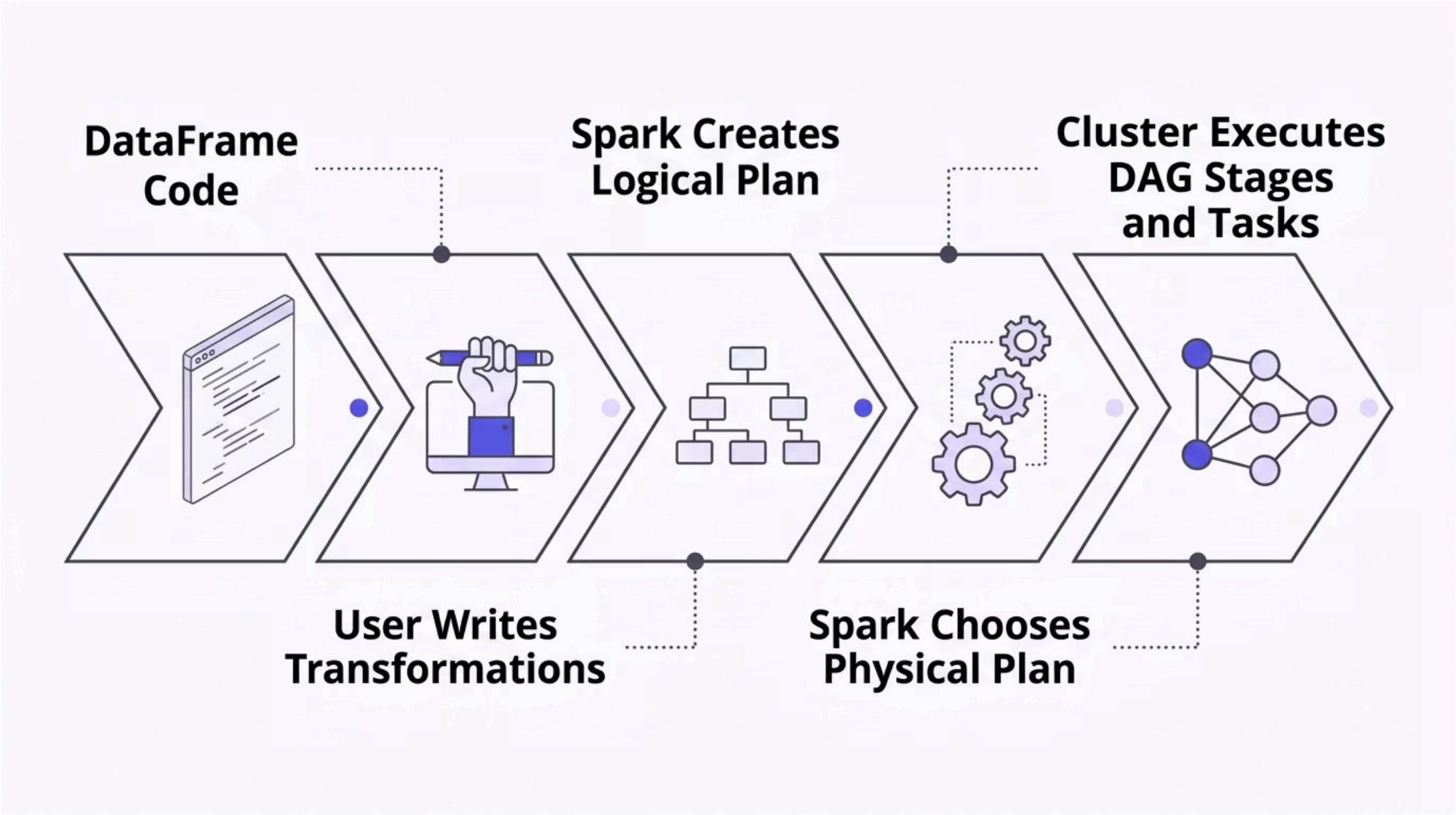
With DataFrames, most operations are **transformations**: they describe how data should be changed, but they do not run immediately. Spark waits until you call an **action** before it computes anything. This design allows the engine to analyse the full sequence of operations and optimise them as one coordinated plan rather than executing each line independently.

Typical transformations include `select`, `filter`, and `withColumn`. Common actions include `show()`, `count()`, `collect()`, and `write`. The practical takeaway is simple: if nothing seems to be happening yet, that is often expected. Spark is still building the plan.

CHAPTER 1 PLANNING

# From Code to DAG

Every DataFrame operation contributes to a chain of logical steps. Spark first builds a **logical plan** that captures *what* you want done. It then derives a **physical plan** that determines *how* the work will actually run, including joins, scans, shuffles, and stage boundaries. These plans are reflected in the DAG, or directed acyclic graph, that Spark executes across the cluster.




This distinction is useful when debugging performance. If results are correct but slow, the issue is often not your business logic itself, but the physical strategy Spark selected to realise it.

CHAPTER 2 OPTIMISATION

# Catalyst Optimiser at a High Level

The Catalyst Optimiser is Spark's rule-based and cost-aware planning engine for structured queries. It inspects the DataFrame plan, applies transformations, and searches for a more efficient equivalent execution strategy. The goal is not to change your result, only to reduce unnecessary work.

### Query analysis

Validates column references, resolves data types, and interprets expressions before execution planning begins.

### Predicate pushdown

Moves eligible filters closer to the data source so fewer rows are read into Spark in the first place.

### Plan refinement

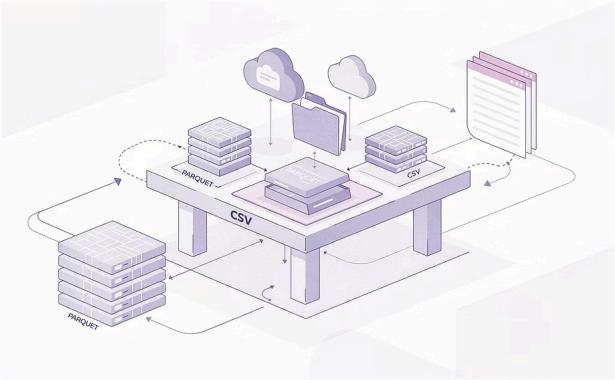
Reorders and simplifies parts of the plan where possible, aiming to minimise scans, shuffles, and intermediate data.

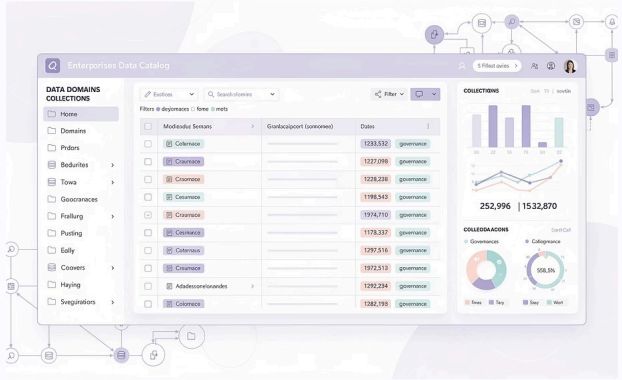
For practitioners, the key lesson is that readable DataFrame code often still performs well because Catalyst can optimise it aggressively—provided the source and operations allow it.

CHAPTER 2 INGESTION

# Creating DataFrames in Databricks

In day-to-day Databricks work, DataFrames are most commonly created from **files** or from **existing tables**. The API stays consistent, but the source matters because formats, metadata, and table definitions influence schema handling, optimisation, and governance.




## From files

Read formats such as Parquet, Delta, CSV, or JSON using Spark readers. File format and options shape how Spark interprets the data.

## From existing tables

Load registered tables directly from the metastore or Unity Catalog, often with stronger discoverability and governance.

> In Databricks, table-backed workflows are often preferred for shared production pipelines, while direct file reads remain common for ingestion, exploration, and raw-zone processing.

CHAPTER 2 RELIABILITY

# Schema Handling

Schema choice is one of the biggest levers for correctness and performance. Spark can **infer** schema from data, which is convenient during exploration, or you can define an **explicit schema**, which is generally safer for production. The trade-off is speed of development versus predictability and control.

## Inference

Fast to start with, especially for ad hoc analysis and notebooks. Useful when the structure is simple and temporary.

Risks include unexpected types, inconsistent parsing, and extra overhead while Spark inspects the data.

## Explicit schema

- Improves type consistency across environments
- Reduces surprises from malformed or sparse records
- Can improve read performance by avoiding inference work
- Best suited to repeatable pipelines and production jobs

A practical default: infer during exploration, then lock the schema down once the pipeline stabilises.

CHAPTER 3 | TRANSFORMATION BASICS

# Column Operations

Column-level transformations are the everyday language of DataFrame work. Two of the most common are `select`, for projecting or reordering columns, and `withColumn`, for creating or replacing derived columns. Together they let you define the shape of a dataset clearly and incrementally.

1

## Select

Choose the columns you want to keep, rename, or compute as part of the projection. This narrows the dataset to what downstream logic actually needs.

2

## withColumn

Add a new column or overwrite an existing one using expressions, functions, or conditional logic built from Spark SQL functions.

3

## Why it matters

Well-scoped column logic improves readability and helps Spark avoid carrying unnecessary data through later stages.

CHAPTER 3 REDUCTION AND SUMMARY

# Filtering and Aggregations

After shaping columns, the next common step is reducing data to the records and summaries that matter. `filter` limits rows based on conditions, while `groupBy` creates grouped aggregations such as counts, sums, averages, or distinct metrics. Together, these operations turn raw events into analytical outputs.

### Filter rows

Apply predicates early to remove irrelevant records and reduce downstream processing volume.

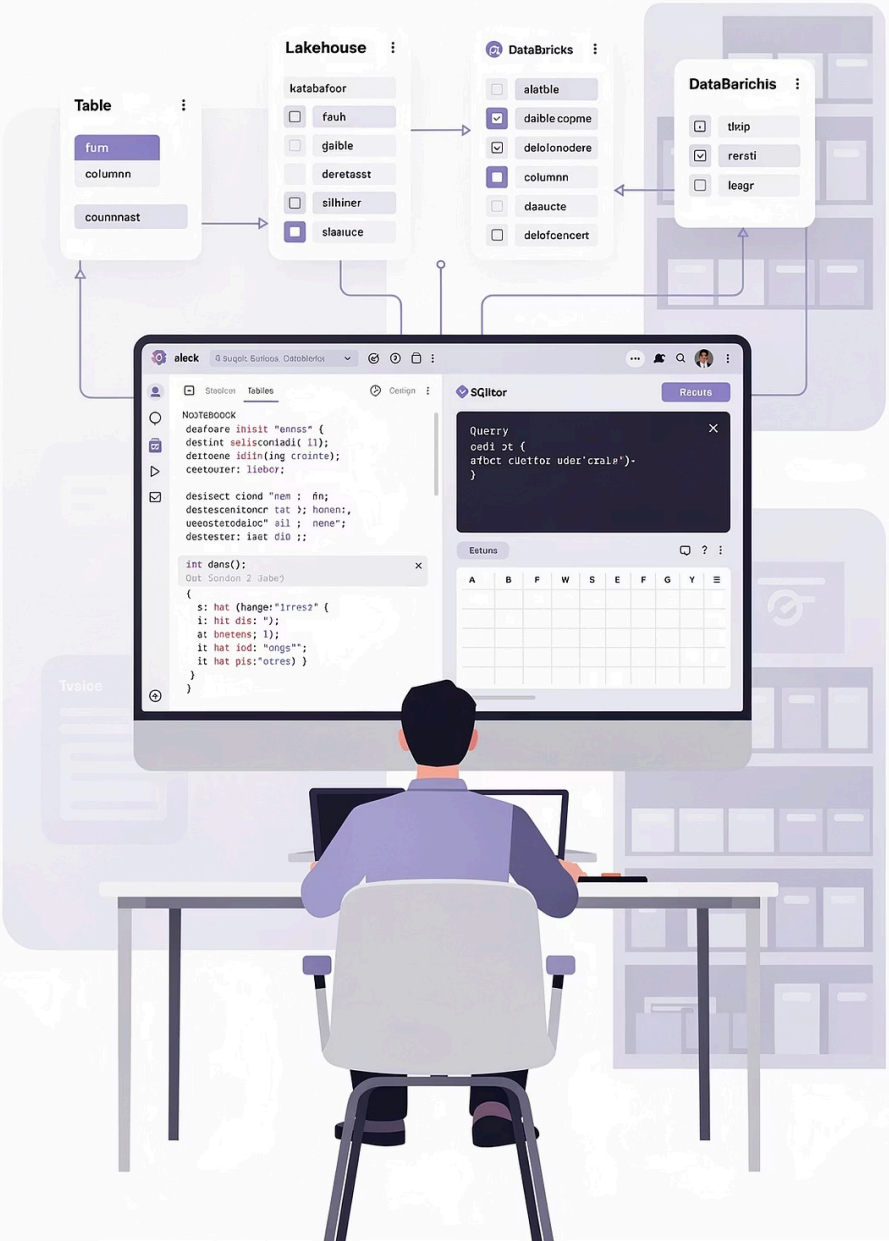
### Group by keys

Organise records by dimensions such as customer, date, region, or model identifier.

### Aggregate values

Compute metrics that can be consumed by dashboards, models, quality checks, or downstream tables.

In performance terms, filtering often reduces cost, while aggregations can introduce shuffles. That is why grouping strategy and data volume still matter even when the API looks simple.




NEXT CHAPTER

SQL-FIRST WORKFLOWS

Everything covered so far carries directly into SQL. The same execution model, planning concepts, optimisation behaviour, schema choices, and transformation patterns still apply; only the interface changes. In the next section, we shift from the DataFrame API to SQL-first workflows and look at how Databricks supports analysts and engineers who prefer expressing logic through queries, views, and table-centric development.

### Same engine

SQL and DataFrames both compile into Spark plans.

### Same optimiser

Catalyst still drives query improvement behind the scenes.

### Different entry point

The workflow becomes more declarative and table-oriented.